

```

1 import numpy as np
2 from math import sqrt
3
4
5 class selfAttention:
6     def __init__(self,dim_in,dim_k,dim_v):
7         self.dim_in,self.dim_k,self.dim_v=dim_in,dim_k,dim_v
8
9         self.W_q=np.random.randn(dim_in,dim_k) * 0.01
10        self.b_q=np.zeros(dim_k)
11
12        self.W_k=np.random.randn(dim_in,dim_k) * 0.01
13        self.b_k=np.zeros(dim_k)
14
15        self.W_v=np.random.randn(dim_in,dim_v) * 0.01
16        self.b_v=np.zeros(dim_v)
17
18        self.W_o=np.random.randn(dim_v,dim_in) * 0.01
19        self.b_o=np.zeros(dim_in)
20
21        self.norm=1/sqrt(dim_k)
22
23    def forward(self,X,mask=None):
24        # X : batch * seq_len * embed
25        Q=X @ self.W_q +self.b_q # B*L*K
26        K=X @ self.W_k +self.b_k # B*L*K
27        V=X @ self.W_v +self.b_v # B*L*V
28
29        score= Q @ K.transpose(0,2,1) *self.norm # B*L*L
30
31        if mask is not None:
32            score=np.where(mask==0,-1e9,score)
33
34        # ä_äö†æ°ä€¼ç¨³ä@šil¼Œä...^â†âŽ»æ¯è¡Œæœ€ä»šä€¼¼
35        score_max=np.max(score,axis=-1,keepdims=True)
36        exp_score=np.exp(score-score_max)
37        weight = exp_score/np.sum(exp_score,axis=-1,keepdims=True)
38
39        atten=weight @ V # B*L*V
40        output=atten @ self.W_o +self.b_o
41        return weight,output
42
43 class MultiHeadAttention:
44     def __init__(self,dim_in,dim_k,dim_v,n_head):
45         self.dim_in,self.dim_k,self.dim_v=dim_in,dim_k,dim_v
46         self.n_head=n_head
47
48         self.W_q=np.random.randn(dim_in,dim_k)*0.01
49         self.b_q=np.zeros(dim_k)
50
51         self.W_k=np.random.randn(dim_in,dim_k)*0.01
52         self.b_k=np.zeros(dim_k)
53
54         self.W_v=np.random.randn(dim_in,dim_v)*0.01
55         self.b_v=np.zeros(dim_v)
56
57         self.W_o=np.random.randn(dim_v,dim_in)*0.01
58         self.b_o=np.zeros(dim_in)
59
60         self.norm=(1/sqrt(dim_k/n_head))
61
62    def forward(self,X,mask=None):
63        batch_size,seq_len,embed=X.shape
64        Q = X @ self.W_q + self.b_q # B*L*(nk)
65        Q = Q.reshape(batch_size,seq_len,self.n_head,-1).transpose(0,2,1,3) # B*n*L*k
66
67        K = X @ self.W_k + self.b_k # B*L*(nk)
68        K = K.reshape(batch_size,seq_len,self.n_head,-1).transpose(0,2,1,3) # B*n*L*k
69
70        V = X @ self.W_v + self.b_v # B*L*(nv)
71        V = V.reshape(batch_size,seq_len,self.n_head,-1).transpose(0,2,1,3) # B*n*L*v
72
73        score = Q @ K.transpose(0,1,3,2) * self.norm # B*n*L*L
74
75        if mask is not None:
76            if mask.ndim==2: # l*L -> B*n*L*L
77                mask = mask[None, None, :, :]
78            elif mask.ndim==3:
79                mask = mask[:,None, :, :]
80            score=np.where(mask==0,-1e9,score)
81
82        score_max=np.max(score,axis=-1,keepdims=True)
83        score_exp=np.exp(score-score_max)
84        weight=score_exp/np.sum(score_exp,axis=-1,keepdims=True)
85        attention=weight @ V # B*n*L*v
86        attention=attention.transpose(0,2,1,3).reshape(batch_size,seq_len,-1) # B*L*V

```

```
87         output= attention @ self.W_o +self.b_o
88
89         return weight,output
90
91
92
93
94
95
96
97
98
99 if __name__ == "__main__":
100     dim_in,dim_k,dim_v=16,8,8
101     batch_size=3
102     seq_len=5
103     n_head=4
104
105     X=np.random.randn(batch_size,seq_len,dim_in)
106     mask=np.random.randn(batch_size,seq_len,seq_len)
107
108     Atten=selfAttention(dim_in,dim_k,dim_v)
109     MultiAtten=MultiHeadAttention(dim_in,dim_k,dim_v,n_head)
110     weight,output = MultiAtten.forward(X,mask)
111     print("Input shape:",X.shape)
112     print("Attention weight shape:",weight.shape)
113     print("Output shape",output.shape)
114
115
116
117
```